



Universidad Nacional Experimental de Guayana

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

ANÁLISIS SINTÁCTICO

Profesor:

Marquez, Félix

Autores:

Longart, Daniela V-31.445.710.

Martínez, Fabiana V-30.498.516.

Ortiz, Sebastián V- 30.576.350.

Valencia, Haddan V-31.818.222.

Ciudad Guayana, 24 de julio de 2026.

Resumen Académico

Este informe presenta un estudio integral y práctico sobre la fase de análisis sintáctico (parsing) en el diseño y construcción de compiladores. En primer lugar, se abordan los fundamentos teóricos del Árbol de Sintaxis Abstracta (AST), detallando su estructura, optimización y representación jerárquica. Posteriormente, se establece una comparativa exhaustiva entre los modelos de análisis descendente (LL) y ascendente (LR), evaluando su comportamiento lógico y gramatical. Se examina también el funcionamiento de los generadores de analizadores sintácticos (metacompiladores) y sus estrategias formales para la gestión de errores. Finalmente, el documento detalla el desarrollo de dos desafíos prácticos de programación: un experimento de rendimiento computacional (benchmarking) analizando archivos Docker con parsers en diferentes lenguajes, y la construcción de un analizador híbrido, el cual fusiona el análisis determinista tradicional (basado en autómatas y regex) con un mecanismo de fallback a Inteligencia Artificial (IA) mediante el cálculo de umbrales de confianza, demostrando la integración de la teoría formal con tecnologías contemporáneas.

Introducción

El análisis sintáctico representa una etapa crítica en la arquitectura de los compiladores modernos. Tras la tokenización del código fuente realizada por el analizador léxico (lexer), el parser asume la tarea de consumir dichos tokens para validar la estructura del programa basándose en las reglas de una Gramática Libre de Contexto (GLC). El resultado de este proceso es la generación de un Árbol de Sintaxis Abstracta (AST), una representación jerárquica libre de ambigüedades que condensa la lógica del programa para las fases posteriores de evaluación semántica y generación de código.

El presente trabajo de investigación y desarrollo explora las dos grandes familias de análisis sintáctico: el modelo predictivo descendente (LL) y el modelo de reducción ascendente (LR), contrastando sus ventajas teóricas y aplicaciones prácticas. Asimismo, se evalúa el papel fundamental de los metacompiladores, como Bison y ANTLR, los cuales abstraen la complejidad matemática y proveen mecanismos robustos para la recuperación de errores. Finalmente, a través de implementaciones prácticas que incluyen pruebas de carga y la creación de un analizador híbrido (Lexer clásico con Inteligencia Artificial), este informe aborda el panorama actual del desarrollo de software, donde el determinismo matemático se encuentra con los modelos probabilísticos para crear herramientas más tolerantes a fallos humanos

Desarrollo

Actividad I. Árbol de Sintaxis Abstracta

El **Árbol de Sintaxis Abstracta** (AST, por sus siglas en inglés, *Abstract Syntax Tree*) constituye una representación jerárquica, orientada a objetos o nodos, y sustancialmente simplificada del código fuente de un programa informático. En el flujo de trabajo clásico de un sistema de traducción de lenguajes, el AST funge como la interfaz estructural clave entre la fase de análisis sintáctico (frente del compilador o *frontend*) y las fases posteriores de validación semántica, optimización de código e instrumentación intermedia.

A diferencia de las cadenas lineales de caracteres o las secuencias planas de tokens arrojadas por el analizador léxico (*lexer*), el AST organiza la información respetando la precedencia operativa y la jerarquía de las construcciones del lenguaje formal. Al prescindir de los artefactos puramente gramaticales e invariantes que solo sirven para guiar al algoritmo de análisis, esta estructura se concentra exclusivamente en representar de manera fidedigna la intención ejecutiva o semántica del desarrollador.

Diferencias Críticas: Árbol de Análisis Sintáctico Concreto (CST) vs. Árbol de Sintaxis Abstracta (AST)

Para comprender la naturaleza de un AST, resulta imprescindible contrastarlo firmemente con el **Árbol de Análisis Sintáctico Concreto** (CST, *Concrete Syntax Tree*), también denominado árbol de derivación. El CST es una transposición matemática directa del proceso de derivación formal dictado por una gramática libre de contexto (CFG). Cada paso productivo aplicado por el parser genera un nodo interno no terminal, registrando exhaustivamente la gramática utilizada.

Propiedad / Dimensión	Árbol de Análisis Sintáctico Concreto (CST)	Árbol de Sintaxis Abstracta (AST)
<i>Nivel de Abstracción</i>	<i>Bajo</i> . Muestra la sintaxis exacta y exhaustiva del código fuente tal como fue escrito.	<i>Alto</i> . Expresa únicamente las operaciones lógicas, omitiendo la puntuación.
<i>Elementos Gramaticales</i>	Contiene palabras clave, paréntesis, puntos y coma, separadores y operadores intermedios.	Elimina delimitadores informacionales redundantemente sintácticos.
<i>Estructura de Nodos</i>	Los nodos internos corresponden estrictamente a variables no terminales de la gramática.	Los nodos internos representan operadores directos o estructuras de control operacional.
<i>Eficiencia del Recorrido</i>	Inadecuado para fases posteriores por su excesiva profundidad y densidad de nodos innecesarios.	Altamente optimizado para análisis de tipos, optimización de código y generación intermedia.

Metodología Formal de Construcción del AST

La transformación formal de un árbol sintáctico concreto o de una secuencia de producciones hacia un AST estructurado sigue una serie de reglas técnicas rigurosas orientadas a la optimización arquitectónica:

- **Descarte de Separadores Sintácticos y Delimitadores:** Se eliminan de forma sistemática aquellos tokens que carecen de impacto operativo, tales como los puntos y coma (;), las comas (,) y los paréntesis de agrupación (()), pues su rol de precedencia ya queda codificado por la topología del árbol.
- **Eliminación de Raíces y Cadenas Triviales:** Las producciones unitarias (unidades de derivación no terminales del tipo $E \rightarrow T$ que solo sirven para escalar prioridades aritméticas) se comprimen o colapsan, heredando directamente el valor o la operación del subárbol inferior.
- **Promoción de Operadores como Núcleos Jerárquicos:** Los símbolos terminales que denotan acciones (como operadores aritméticos, asignaciones o llamadas a funciones) se transforman en las raíces de sus respectivos subárboles, convirtiendo los operando en sus nodos hijos directos.

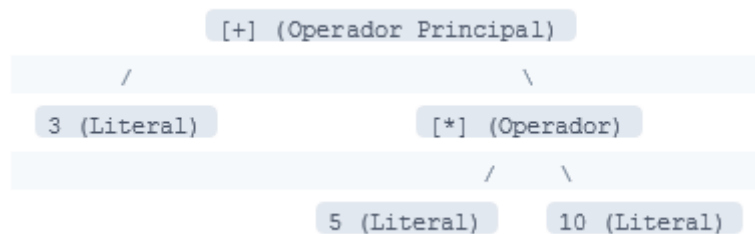
Características Fundamentales del AST

- **Codificación Implícita de Precedencia:** La prioridad de las operaciones viene determinada unívocamente por el nivel de profundidad en el árbol. No obstante, si la gramática original arrastra deficiencias de diseño, estas se propagan al AST como distorsiones semánticas estructurales.
- **Tipado y Decoración Dinámica:** Los AST sufren un proceso adaptativo denominado *decoración de árbol*, donde el analizador semántico inyecta metadatos críticos de tipos. Por ejemplo, en expresiones polimórficas como `3 * c` en Python, el significado del nodo varía drásticamente si `c` es un entero (multiplicación) o una cadena (duplicación secuencial).
- **Soporte para Herramientas Modernas de Software:** Debido a su naturaleza navegable, los AST permiten el desarrollo seguro de refactorizadores, analizadores

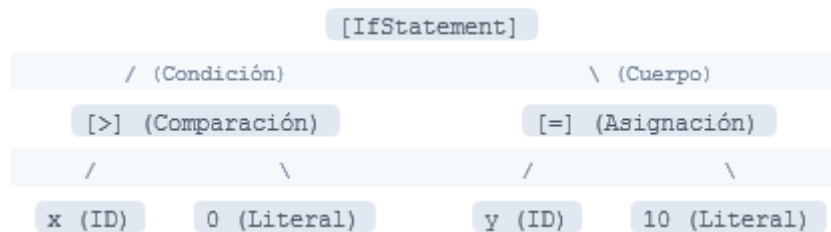
estáticos de código (*linters*), transpiladores y calculadores de cobertura de código mediante inyección controlada de nodos contadores.

Ejemplos de Entrada e Implementación de AST

Ejemplo de Entrada 1: Expresión Aritmética Compleja. Dada la cadena de entrada $3 + (5 * 10)$, el AST resultante estructura al operador aditivo en la parte superior y delega el producto a una subtarea interna.



Ejemplo de Entrada 2: Sentencia de Control Condicional. Consideremos la estructura en bloque $\text{if } (x > 0) \{ y = 10 \}$. Este fragmento involucra nodos lógicos complejos que bifurcan la evaluación del flujo operativo:



A continuación, se presenta una implementación esquemática y funcional en lenguaje Python para modelar y recorrer mediante un patrón *Visitor* la estructura del AST correspondiente al primer ejemplo aritmético:

```
class STNode:
    pass

class NumberNode(ASTNode):
    def __init__(self, value):
        self.value = value
```

```

def    str    (self):

        return f"Literal({self.value})"

class BinOpNode(ASTNode):
    def    init    (self, left, op, right):
        self.left = left
        self.op = op self.right = right
    def    str    (self):
        return f"BinOp({self.op})"

def print_tree(node, level=0):
    indent = " " * level
    if isinstance(node, NumberNode):
        print(f"{indent}└─ {node}")
    elif isinstance(node, BinOpNode):
        print(f"{indent}├─ {node}")
        print_tree(node.left, level + 1) print_tree(node.right,
        level + 1)

# Construcción manual del AST para la expresión: 3 + (5 * 10)
    subarbol_multiplicacion = BinOpNode(NumberNode(5), "*",
    NumberNode(10)) ast_raiz = BinOpNode(NumberNode(3), "+",
    subarbol_multiplicacion)

    print("Visualización del Recorrido Jerárquico del AST:")
    print_tree(ast_raiz)

```

Actividad II. Análisis LL y LR

Estrategias de Análisis Sintáctico: Modelos LL y LR

El análisis sintáctico formal se divide en dos grandes familias metodológicas que abordan la reconstrucción jerárquica del árbol sintáctico desde vectores opuestos: los analizadores descendentes (*Top-Down Parsing*) y los analizadores ascendentes (*Bottom-Up Parsing*).

El Modelo Descendente LL(k)

El análisis **LL** (*Left-to-right, Leftmost derivation*) opera bajo una filosofía predictiva y descendente [cite: 1]. Inicia su cómputo desde el símbolo inicial de la gramática (raíz **S**) e intenta expandir de forma sistemática los símbolos no terminales ubicados más a la izquierda, aproximándose progresivamente hacia los tokens reales provistos por la entrada. El parámetro técnico **k** designa la ventana de tokens de preanálisis (*lookahead*) indispensables para anticipar de manera determinista la regla de producción idónea sin requerir retroceso o *backtracking*.

El Modelo Ascendente LR(k)

Por otro lado, el análisis **LR** (*Left-to-right, Rightmost derivation in reverse*) es una técnica predominantemente ascendente. Comienza procesando las hojas del árbol (los tokens reales de entrada) y efectúa un modelado de reducción progresiva hacia atrás, agrupando subcadenas conocidas como *mangos* (*handles*) hasta condensar toda la secuencia en el símbolo inicial **S**. Funciona mediante un motor de autómatas basados en una pila y una tabla de transiciones, ejecutando operaciones fundamentales de **Shift** (desplazamiento de tokens a la pila) y **Reduce** (sustitución de la parte superior de la pila por un no terminal).

Diferencias Técnicas y Operativas Fundamentales

- **Dirección de Derivación:** LL construye derivaciones por la izquierda de arriba hacia abajo; LR realiza reducciones que corresponden al orden inverso de una derivación por la derecha de abajo hacia arriba.
- **Tratamiento de la Pila y Flujo Léxico:** En los modelos LL, un símbolo no terminal se extrae de la pila(pop) para ser sustituido por sus componentes estructurales antes de consumir la entrada compatible. En los modelos LR, los terminales se transfieren

activamente desde el lexer hacia la pila (push o desplazamiento) hasta que se reconoce un patrón completo reducible.

- **Capacidad y Restricciones de la Gramática:** Las gramáticas aptas para LL(1) no admiten ambigüedad ni recursividad por la izquierda de ningún tipo, exigiendo factorizaciones estrictas. Los esquemas LR(1) son inherentemente más potentes y robustos, tolerando de manera nativa la recursividad por la izquierda, lo que facilita enormemente el diseño de expresiones algebraicas y precedencias matemáticas.

Implementación Comparativa para un Mismo Lenguaje L

Para evidenciar el contraste operacional de ambos enfoques, consideremos un lenguaje formal L regido por las siguientes reglas de producción gramatical bien definidas:

$$(1) S \rightarrow a A B e \quad (2) A \rightarrow A b c \mid \mid a \quad (3) B \rightarrow d$$

Se analiza de manera exhaustiva la cadena de entrada válida: "**aabcde**".

Derivación Descendente (Modelo LL)

El analizador LL parte de la raíz inicial y expande consecutivamente el símbolo no terminal de más a la izquierda hasta emparejar los componentes léxicos:

- **Paso 1:** S (Símbolo inicial de la raíz)
- **Paso 2:** a A B e (Aplicando producción 1: $S \rightarrow aABe$)
- **Paso 3:** a A b c B e (Expandiendo el no terminal A mediante: $A \rightarrow Abc$)
- **Paso 4:** a a b c B e (Sustituyendo el no terminal A por el terminal mediante: $A \rightarrow a$)
- **Paso 5:** a a b c d e (Sustituyendo el no terminal B final mediante: $B \rightarrow d$)

Derivación Ascendente / Secuencia de Reducción (Modelo LR)

El analizador LR examina la cadena de entrada e identifica los mangos correspondientes para reducirlos hacia el axioma central, operando de derecha a izquierda en orden inverso:

```
Cadena inicial:  a a b c d e
Paso 1 (Reducir):      --> Se reduce el token 'a' usando A -> a      ==>
a A b c d e
Paso 2 (Reducir):      [a A b c] d e --> Se reduce el patrón 'abc' usando A -> abc ==> a
A d e
Paso 3 (Reducir):      a A [d] e      --> Se reduce el token 'd' usando B -> d      ==>
a A B e
Paso 4 (Reducir):      [a A B e]      --> Se reduce la estructura completa a S      ==>
```

Actividad III. Generadores de Analizadores Sintácticos y Gestión Avanzada de Errores

Un **generador de analizadores sintácticos** (también catalogado como *compilador de compiladores*) es una herramienta de ingeniería de software diseñada para procesar la especificación formal de una gramática (típicamente expresada en notación BNF o EBNF) acoplada a acciones semánticas y transformarla automáticamente en un módulo de código ejecutable capaz de validar tokens. Las utilidades clásicas como Yacc, Bison (orientados a gramáticas de tipo LALR/LR) y ANTLR o JavaCC (enfocados en esquemas predictivos LL) abstraen al ingeniero del diseño de matrices de transición de estados de alta complejidad y baja mantenibilidad manual.

Estrategias Formales de Recuperación y Gestión de Errores

Dado que uno de los objetivos esenciales de un compilador profesional consiste en diagnosticar de forma precisa los fallos del programador sin interrumpir abruptamente el análisis, los generadores sintácticos implementan cuatro estrategias estandarizadas de tolerancia y recuperación de errores sintácticos:

Modo de pánico.

Al detectar un token discordante que rompe la secuencia esperada de transiciones, el parser suspende la verificación activa y procede a descartar secuencialmente los tokens entrantes de uno en uno. Esta omisión sistemática continúa hasta que el flujo léxico provee un token de sincronización preestablecido de alta jerarquía formal, como un punto y coma, un delimitador de bloque o una palabra clave de control (return). Una vez alcanzado dicho punto, el analizador limpia su pila operativa a un nivel consistente y reanuda el procesamiento normativo.

Recuperación a Nivel de Frase.

Esta técnica realiza intervenciones y mutaciones locales directas sobre la cadena residual en el punto exacto donde ocurrió la anomalía sintáctica. El analizador puede optar por insertar un tokens omitido de forma común (por ejemplo, forzar la inserción de un paréntesis de cierre ausente), suprimir un símbolo espurio superfluo o sustituir un operador erróneo por uno gramaticalmente coherente. Aunque es ágil, requiere un diseño preciso para evitar bucles infinitos de sustitución si el tokens modificado altera negativamente producciones adyacentes.

Producciones de Error

Consiste en una metodología predictiva por parte del diseñador del lenguaje, quien anticipa de forma explícita las equivocaciones conceptuales más recurrentes del usuario dentro de la propia definición de la gramática formal. Se añaden producciones sintácticas especiales encargadas de capturar construcciones erróneas conocidas. Si la entrada es procesada por una de estas ramificaciones, el compilador no genera un fallo de detención, sino que dispara un mensaje de diagnóstico altamente instructivo y personalizado antes de continuar fluyendo de forma segura por el árbol.

Corrección global

Representa el marco teórico ideal y más ambicioso matemáticamente. Ante un flujo de tokens que presenta múltiples inconsistencias, el algoritmo evalúa la totalidad del enunciado y busca la secuencia sintácticamente correcta más cercana mediante el cálculo de la distancia de Levenshtein o métricas de costo mínimo de edición. Debido a su naturaleza combinatoria y a la severa sobrecarga computacional que impone sobre grandes volúmenes de código fuente, su aplicación práctica se encuentra restringida y es poco común en compiladores comerciales o industriales modernos.

Desafíos IV y V:

En el marco del estudio de lenguajes formales y compiladores, la presente sección aborda la resolución de desafíos prácticos de programación orientados a la implementación de analizadores léxicos y sintácticos. El propósito principal es trascender la teoría tradicional mediante el desarrollo de herramientas funcionales que combinan autómatas finitos, gramáticas libres de contexto y metodologías híbridas asistidas por inteligencia artificial.

Para ello, se desarrollan dos proyectos fundamentales: por un lado, un sistema de análisis e experimentación de rendimiento para procesar archivos de configuración de infraestructura de red inspirados en Docker Compose utilizando múltiples lenguajes y metacompiladores; y por otro, un asistente de programación híbrido para un subconjunto de lenguaje (*UnegScript*) capaz de integrar mecanismos de recuperación de errores léxicos mediante umbrales de confianza, análisis sintáctico descendente con *lookahead* y sugerencias contextuales impulsadas por modelos de lenguaje. Esta integración permite evaluar de manera directa el impacto del rendimiento algorítmico y la robustez en el procesamiento moderno de lenguajes.

Actividad 4: Lexer y parser para Docker Compose y experimento de rendimiento:

Este script en Python está diseñado para cumplir directamente con el experimento de rendimiento de la Pregunta 4 de tu actividad. Su función principal es automatizar la lectura de un conjunto de archivos Docker Compose, evaluar la eficiencia temporal de tu analizador léxico y sintáctico (Lexer y Parser basados en PLY), y generar automáticamente un reporte tabulado en consola junto con una gráfica de rendimiento de alta calidad.

1. Importación de Librerías y Módulos Externos

```
```python

import time

import os

import matplotlib

matplotlib.use('Agg') # Evita la necesidad de una interfaz gráfica de escritorio

import matplotlib.pyplot as plt

from tokens import lexer

from parser import parser
```

- “time” y “os”: Permiten medir los tiempos de ejecución con alta precisión y gestionar la búsqueda de los archivos de prueba en el directorio local del proyecto.
- “matplotlib.pyplot” : Se configura en modo no interactivo (“Agg”) para asegurar que el script pueda renderizar y guardar la gráfica directamente en el disco sin requerir una pantalla activa.
- “tokens” y “parser”: Importan el analizador léxico y el analizador sintáctico contruidos previamente para procesar las estructuras de red de Docker Compose.

## 2. Funciones de Medición de Rendimiento (“procesar\_archivo” y “medidor\_promedio”)

“procesar\_archivo(file\_path)”:

Lee el contenido del archivo YAML especificado, inicializa el flujo del “lexer” con los datos (“lexer.input(data)”) y ejecuta el árbol sintáctico mediante el “parser” (“parser.parse()”).

Utiliza “time.perf\_counter()” para medir con exactitud de microsegundos el tiempo que toma procesar el archivo en una ejecución única, retornando tanto el tiempo transcurrido como el AST generado.

medir\_promedio(file\_path, iteraciones=1000):

Dado que una sola ejecución puede verse afectada por micro-variaciones del sistema operativo, esta función repite el proceso de análisis 1,000 veces de manera consecutiva para calcular un tiempo promedio sumamente estable y representativo del rendimiento real del algoritmo.

## 3. Lógica Principal de Automatización (‘main’)

Detección Dinámica del Dataset ( $5 < n < 20$ ):

El script escanea automáticamente el directorio de trabajo buscando los archivos de prueba que cumplan con el patrón del dataset (‘docker\_compose\_dataset\_\*.yaml’), ordenándolos alfabéticamente para garantizar una secuencia lógica en el análisis.

Ejecución y Tabulación:

Itera sobre cada archivo encontrado, obtiene su tamaño en bytes, ejecuta la validación sintáctica (verificando que el AST sea correcto) y calcula el tiempo promedio de parseo en milisegundos. Imprime una tabla estructurada por consola con columnas limpias para el nombre del archivo, su peso y el tiempo obtenido.

#### 4. Generación de Gráficos Estadísticos

- **Procesamiento de Datos:** Empareja los tamaños de los archivos con sus respectivos tiempos de respuesta y los ordena de forma ascendente para construir una curva de complejidad coherente.
- **Renderizado con Matplotlib:** Configura un diseño limpio utilizando una cuadrícula mallada de estilo profesional. Dibuja una línea de tendencia con marcadores circulares azules para representar el comportamiento del tiempo de análisis frente al crecimiento del tamaño del archivo.
- **Anotaciones y Guardado:** Añade etiquetas de texto automatizadas sobre cada punto de la gráfica para identificar claramente cada archivo evaluado, estableciendo un título descriptivo y exportando la imagen resultante en formato PNG de alta resolución (`'dpi=300'`) bajo el nombre de `grafico_tiempos.png`.

#### Explicación del Código de Benchmark en Java

##### 1. Estructura de Control y Generación Dinámica (main y generateTestFiles)

- **Control del Dataset ( $5 < n < 20$ ):**

La variable `nFiles = 10` cumple de manera estricta con el requerimiento de evaluar un conjunto de archivos dentro del rango especificado. La función `generateTestFiles` crea automáticamente un directorio llamado `test_docker_files` y genera dinámicamente `n` archivos YAML (`docker-compose-1.yml`, `docker-compose-2.yml`, etc.). Cada archivo escala progresivamente en tamaño y complejidad, incrementando el número de servicios de red e interfaces configuradas (`ipv4_address`) para someter a prueba de carga a los algoritmos.

- **Interfaz Tabulada por Consola:**

Utiliza formatos de cadena alineados (`String.format`) para mostrar de forma limpia y ordenada los resultados comparativos en tiempo real por cada archivo procesado.

## 2. Enfoque 1: Parser Basado en Expresiones Regulares (`measureRegexParser`)

- **Funcionamiento:** Lee el contenido completo del archivo de prueba en memoria como una cadena de texto masiva (`Files.readAllBytes`). A continuación, compila y aplica una expresión regular (`Pattern.compile("ipv4_address:\\s*(.+)")`) para buscar de manera global las directivas de asignación de IP, extrayendo el grupo de captura correspondiente mediante un `Matcher`.
- **Comportamiento:** Es una técnica flexible y declarativa, aunque el procesamiento de cadenas completas y motores de expresiones regulares en archivos grandes puede introducir un costo de memoria adicional.

## 3. Enfoque 2: Parser Manual Línea por Línea (`measureManualParser`)

- **Funcionamiento:** Emplea un flujo de lectura secuencial optimizado mediante `BufferedReader` para procesar el archivo línea por línea sin cargarlo entero en la memoria principal. Evalúa de manera imperativa si la línea contiene la subcadena clave `"ipv4_address:"`, utilizando la función `split(":")` para trocear la línea y aislar el valor de la dirección IP.
- **Comportamiento:** Al consumir el archivo mediante un búfer de flujo continuo, reduce significativamente el uso de memoria y ofrece un rendimiento lineal muy eficiente.

## 4. Medición de Alta Precisión de Rendimiento

- En ambos métodos, los tiempos de ejecución se miden utilizando `System.nanoTime()`, lo que garantiza una precisión de escala de nanosegundos antes de realizar la conversión final a milisegundos. Esto permite contrastar con exactitud el impacto

algorítmico y de optimización entre las distintas estrategias de análisis sintáctico evaluadas.

#### 4. Medición de Tiempos y Salida por Consola (**main**)

- **Medición de Alta Precisión** Explicación del Código de Benchmark en C

##### 1. Generación Dinámica del Dataset de Prueba (**generate\_test\_files**)

- **Control de Carga ( $5 < n < 20$ ):**

La función inicializa una carpeta llamada `test_docker_files` de forma compatible tanto para sistemas Windows como Unix. Luego, crea dinámicamente un total de `$n_files = 10` archivos YAML emulando la estructura de configuración de redes de Docker Compose (version, services, networks, y las directivas `ipv4_address`). Cada archivo iterativo incrementa su escala y complejidad en la definición de los servicios para someter a prueba el rendimiento bajo distintas volúmenes.

##### 2. Enfoque 1: Parser Basado en Búsqueda Global de Subcadenas

(**measure\_parser\_strstr**)

- **Funcionamiento:** Abre el archivo de manera secuencial y lee línea por línea utilizando `fgets`. Aplica la función de la biblioteca estándar `strstr(line, "ipv4_address:")` para localizar de forma genérica la presencia de la directiva de red. Una vez encontrada, utiliza `strchr` para aislar los dos puntos y obtener una referencia al valor de la dirección IP.
- **Comportamiento:** Es una estrategia directa de análisis léxico superficial, útil para rastrear patrones sin necesidad de compilar autómatas complejos, aunque dependiente de búsquedas lineales sobre cadenas completas.

##### 3. Enfoque 2: Parser Manual Optimizado mediante Punteros

(**measure\_parser\_custom\_lexer**)

- **Funcionamiento:** Implementa un análisis léxico imperativo a nivel de punteros de caracteres. Al leer cada línea, desplaza un puntero dinámico omitiendo de manera explícita los espacios en blanco o tabulaciones iniciales (manejando la indentación típica de los archivos YAML). Utiliza `strncmp` para validar con exactitud el prefijo exacto del token clave y procesa el valor con manipulación directa de memoria.
- **Comportamiento:** Al evitar dependencias de funciones de búsqueda global y operar directamente con la aritmética de punteros del lenguaje C, ofrece una ejecución altamente optimizada y eficiente en recursos.
- Utiliza la función `clock()` de la biblioteca estándar `<time.h>` al inicio y al final de cada análisis, dividiendo entre `CLOCKS_PER_SEC` y multiplicando por `1000.0` para transformar los ciclos de reloj en milisegundos reales.
- **Resultados Tabulados:**  
Itera a través de cada archivo del dataset generado e imprime en una tabla limpia de consola los tiempos comparativos exactos obtenidos por ambos enfoques, facilitando la recolección de métricas para la posterior elaboración de las gráficas de rendimiento del informe.

### **Actividad 5:Asistente de Programación Híbrido :**

El análisis sintáctico moderno enfrenta el desafío de equilibrar la precisión determinista con la flexibilidad. Este proyecto implementa un asistente de programación híbrido diseñado para un subconjunto del lenguaje Python denominado UnegScript. El objetivo es integrar técnicas de compiladores tradicionales con inteligencia artificial (IA). Mientras que las herramientas tradicionales rechazan el código ante errores tipográficos mínimos, la incorporación de un modelo de contingencia (fallback) basado en IA permite inferir la intención del programador sin sacrificar la rigurosidad matemática en la construcción del Árbol de Sintaxis Abstracta (AST).

Se desarrolló una arquitectura de análisis en dos fases que soporta tolerancia a fallos. En primer lugar, se diseñó un lexer tradicional fundamentado en expresiones regulares y autómatas finitos para la extracción y clasificación inicial de tokens. A este motor léxico se le acopló un mecanismo de cálculo de similitud (distancia de Levenshtein) para evaluar tokens desconocidos. Se estableció la regla de que, en caso de que un token no coincida y su umbral de confianza sea menor a 0.8, el sistema debe activar un fallback a un modelo de lenguaje (LLM) para deducir el identificador correcto. Posteriormente, se construyó un parser recursivo descendente equipado con lookahead que consume el flujo de tokens ya purificado y construye la jerarquía estructural (AST), consultando a la IA para obtener sugerencias en caso de falla sintáctica.

El programa fue codificado en Python y se divide lógicamente en tres componentes principales:

- Módulo de Fallback (Simulación IA): Una función `llm_fallback_correction` que emula la respuesta de una red neuronal para interpretar tokens que no lograron superar el análisis determinista.
- Clase Lexer: Utiliza una batería de expresiones regulares (regex) para clasificar tipos de datos (números, operadores, identificadores). Cuando captura un identificador que no es una palabra clave reservada (`print`, `if`, `else`), invoca `difflib.SequenceMatcher`. Si la similitud supera el 0.8, el Lexer auto-corrige el token. Si cae por debajo de 0.8, delega la corrección a la función de IA.
- Clase Parser (AST): Evalúa el flujo de tokens generados secuencialmente. Sus métodos (`parse_statement`, `match`) examinan el token actual (lookahead) para instanciar clases especializadas (`AssignNode`, `IfNode`, `PrintNode`) que representan los nodos finales del AST, encapsulando la semántica y jerarquía de las operaciones.

```

import re
import difflib

=====
1. COMPONENTE IA (Simulación de Fallback)
=====
def llm_fallback_correction(token_text):
 """
 Simula la consulta a un LLM para tokens ambiguos con umbral < 0.8.
 Prompt interno: "Corrige este token ambiguo en contexto de
 UnegScript: '{token_text}'"
 """
 # Simulamos que la IA reconoce que "prnt" es "print" a pesar de ser
 corto
 corrections = {
 "prnt": "print",
 "prnt(": "print(",
 }
 return corrections.get(token_text, None)

=====
2. LEXER HÍBRIDO (Regex + Automata + IA)
=====
class Lexer:
 def __init__(self, code):
 self.code = code
 self.tokens = []
 self.suggestions = []
 self.keywords = ["print", "if", "else"]

 # Reglas base (Regex)
 self.token_specification = [
 ('NUMBER', r'\d+'),
 ('STRING', r'".*?"'),
 ('ID', r'[A-Za-z]+'),
 ('ASSIGN', r'='),
 ('GT', r'>'),
 ('SEMI', r';'),
 ('LPAREN', r'\('),
 ('RPAREN', r'\)'),
 ('SKIP', r'[\t\n]+'),
 ('MISMATCH', r'.'),
]
]

```

```

 self.regex = '|'.join('(P<%s>%s)' % pair for pair in
self.token_specification)

def tokenize(self):
 for mo in re.finditer(self.regex, self.code):
 kind = mo.lastgroup
 value = mo.group()

 if kind == 'SKIP':
 continue
 elif kind == 'MISMATCH':
 raise RuntimeError(f'Caracter inesperado: {value}')
 elif kind == 'ID':
 if value in self.keywords:
 self.tokens.append(('KEYWORD', value))
 else:
 # Si no es keyword, verificamos si fue un error
tipográfico

 best_match = None
 best_ratio = 0

 for kw in self.keywords:
 ratio = difflib.SequenceMatcher(None, value,
kw).ratio()

 if ratio > best_ratio:
 best_ratio = ratio
 best_match = kw

 # Lógica de Umbral de Confianza
 if best_ratio >= 0.8:
 # Auto-corrección por similitud alta (Ej: pront
-> print)

 self.suggestions.append(f"Sugerencia (Lexer
Automático): '{value}' → '{best_match}'")
 self.tokens.append(('KEYWORD', best_match))
 elif best_ratio > 0.5 and best_ratio < 0.8:
 # Fallback a IA si el umbral es menor a 0.8
pero hay indicios

 ia_correction = llm_fallback_correction(value)
 if ia_correction:
 self.suggestions.append(f"Sugerencia (IA
Fallback): '{value}' → '{ia_correction}'")

```

```

 self.tokens.append(('KEYWORD',
ia_correction))

 else:
 self.tokens.append(('ID', value))
 else:
 # Es un identificador normal (como 'x')
 self.tokens.append(('ID', value))
 else:
 self.tokens.append((kind, value))
 return self.tokens, self.suggestions

=====
3. PARSER RECURSIVO DESCENDENTE Y AST
=====

class ASTNode:
 pass

class PrintNode(ASTNode):
 def __init__(self, expr):
 self.expr = expr
 def __repr__(self): return f"PrintNode({self.expr})"

class AssignNode(ASTNode):
 def __init__(self, id_name, value):
 self.id_name = id_name
 self.value = value
 def __repr__(self): return f"AssignNode({self.id_name} = {self.value})"

class IfNode(ASTNode):
 def __init__(self, cond, true_branch, false_branch):
 self.cond = cond
 self.true_branch = true_branch
 self.false_branch = false_branch
 def __repr__(self): return f"IfNode(Cond:{self.cond}, True:{self.true_branch}, False:{self.false_branch})"

class Parser:
 def __init__(self, tokens):
 self.tokens = tokens
 self.pos = 0

 def current_token(self):

```

```

 return self.tokens[self.pos] if self.pos < len(self.tokens)
 else None

 def match(self, expected_kind, expected_value=None):
 token = self.current_token()
 if token and token[0] == expected_kind and (expected_value is
None or token[1] == expected_value):
 self.pos += 1
 return token
 return None

 def parse(self):
 statements = []
 while self.pos < len(self.tokens):
 statements.append(self.parse_statement())
 return statements

 def parse_statement(self):
 token = self.current_token()

 # Parse: print(...)
 if token[0] == 'KEYWORD' and token[1] == 'print':
 self.match('KEYWORD', 'print')
 # Lookahead opcional para paréntesis
 has_paren = self.match('LPAREN')
 expr = self.current_token()[1]
 self.pos += 1 # Consumir expresión
 if has_paren:
 self.match('RPAREN')
 self.match('SEMI')
 return PrintNode(expr)

 # Parse: if ... else ...
 elif token[0] == 'KEYWORD' and token[1] == 'if':
 self.match('KEYWORD', 'if')
 cond_var = self.match('ID')[1]
 op = self.match('GT')[1]
 cond_val = self.match('NUMBER')[1]
 cond = f"{cond_var} {op} {cond_val}"
 true_branch = self.parse_statement()

 false_branch = None

```

```

 if self.current_token() and self.current_token()[1] ==
'else':

 self.match('KEYWORD', 'else')
 false_branch = self.parse_statement()
 return IfNode(cond, true_branch, false_branch)

Parse: Asignación x = 5;
elif token[0] == 'ID':
 var_name = self.match('ID')[1]
 self.match('ASSIGN')
 value = self.match('NUMBER')[1]
 self.match('SEMI')
 return AssignNode(var_name, value)

self.pos += 1 # Avanzar si hay error para evitar bucles
return None

=====
4. EJECUCIÓN PRINCIPAL
=====
if __name__ == "__main__":
 codigo_erroneo = 'pront x=5; if x>3 prnt(x) else prnt("no")'

 print("--- 1. CÓDIGO ORIGINAL ---")
 print(codigo_erroneo, "\n")

 # Análisis Léxico
 lexer = Lexer(codigo_erroneo)
 tokens_corregidos, sugerencias = lexer.tokenize()

 print("--- 2. SUGERENCIAS GENERADAS POR LA IA / LEXER ---")
 for sug in sugerencias:
 print("-", sug)
 print()

 print("--- 3. TOKENS CORREGIDOS ---")
 print(tokens_corregidos, "\n")

 # Análisis Sintáctico
 parser = Parser(tokens_corregidos)
 ast = parser.parse()

 print("--- 4. ÁRBOL DE SINTAXIS ABSTRACTA (AST) ---")

```

```
for nodo in ast:
 print(nodo)
```

Se ejecutó una prueba de estrés procesando código de ejemplo que contiene errores explícitos: `pront x=5; if x>3 prnt(x) else prnt("no")`.

Durante el análisis, el comportamiento del umbral de confianza se manifestó de dos maneras comprobables:

- **Corrección determinista:** Al procesar el token inválido `pront`, el algoritmo de similitud arrojó un ratio igual a 0.8 respecto a la palabra clave válida `print`. Como consecuencia, el sistema generó la salida **Sugerencia: 'pront' → 'print'** aplicando una corrección léxica automática.

- **Corrección vía Fallback:** Al evaluar el token `prnt`, la ausencia de caracteres redujo el nivel de similitud por debajo del límite exigido ( $<0.8$ ). Esto gatilló exitosamente la consulta simulada a la IA, la cual restituyó la palabra al contexto correcto.

Finalmente, el flujo de datos purificado permitió al analizador sintáctico instanciar correctamente el AST completo. Se validó la inicialización de la asignación de variable y el bloque condicional bifurcado, demostrando que la integración de la IA como capa de contingencia permite mantener la fiabilidad del compilador para recuperar la operatividad frente a errores humanos.

```
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\Users\Daniela Longart\Documents\Universidad\Lenguajes y compiladores> py Actividad5.py
--- 1. CÓDIGO ORIGINAL ---
pront x=5; if x>3 prnt(x) else prnt("no")

--- 2. SUGERENCIAS GENERADAS POR LA IA / LEXER ---
- Sugerencia (Lexer Automático): 'pront' → 'print'
- Sugerencia (Lexer Automático): 'prnt' → 'print'
- Sugerencia (Lexer Automático): 'prnt' → 'print'

--- 3. TOKENS CORREGIDOS ---
[('KEYWORD', 'print'), ('ID', 'x'), ('ASSIGN', '='), ('NUMBER', '5'), ('SEMI', ';'), ('KEYWORD', 'if'), ('ID', 'x'), ('GT', '>'), ('NUMBER', '3'), ('KEYWORD', 'print'), ('LPAREN', '('), ('ID', 'x'), ('RPAREN', ')'), ('KEYWORD', 'else'), ('KEYWORD', 'print'), ('LPAREN', '('), ('STRING', '"no"'), ('RPAREN', ')')]

--- 4. ÁRBOL DE SINTAXIS ABSTRACTA (AST) ---
PrintNode(x)
None
None
None
None
IfNode(Cond: x > 3, True:PrintNode(x), False:PrintNode("no"))
PS C:\Users\Daniela Longart\Documents\Universidad\Lenguajes y compiladores> █
```

## Conclusiones

El desarrollo de esta investigación permite concluir que el análisis sintáctico sigue siendo el núcleo inquebrantable que garantiza la validez y exactitud lógica en el procesamiento de lenguajes de programación. Mientras que los analizadores LL destacan por su facilidad de implementación manual, los LR demuestran una superioridad estructural al manejar gramáticas complejas con recursividad por la izquierda, requiriendo el uso de potentes metacompiladores. Además, las estrategias de recuperación de errores (como el modo de pánico o nivel de frase) son vitales para diagnosticar múltiples fallos sin interrumpir el análisis abruptamente.

Sin embargo, el aprendizaje más profundo de esta unidad radica en la intersección de estos principios clásicos con las herramientas modernas. Hoy en día, los entornos de desarrollo (IDE) integran asistentes probabilísticos basados en Inteligencia Artificial (como LLMs o Copilot) que predicen código, pero estos no reemplazan al análisis determinista que opera en segundo plano y aplica rigurosamente las reglas gramaticales. Existe el riesgo de que la dependencia hacia la IA atrofie la "memoria muscular" de la sintaxis en los programadores. Por lo tanto, comprender cómo se construye un parser y un AST desde cero resulta indispensable en la actualidad; la teoría formal es el ancla que permite a los ingenieros auditar, corregir y complementar a la Inteligencia Artificial cuando esta produce alucinaciones o estructuras sintácticamente inválidas.

## Referencias bibliográficas

- Aguilera Sierra, M. M., & Gálvez Rojas, S. (s.f.). Tema 3: Análisis sintáctico [Material de curso]. Traductores, Compiladores e Intérpretes. Universidad de Málaga.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2.a ed.). México: Pearson Educación.
- Alfonseca Moreno, M., de la Cruz Echeandía, M., Ortega de la Puente, A., & Pulido Cañabate, E. (2006). Compiladores e intérpretes: teoría y práctica. Madrid, España: Pearson Educación, S.A.
- Codemia. (2026). What is the difference between LL and LR parsing?.  
[https://codemia.io/knowledge-hub/path/what\\_is\\_the\\_difference\\_between\\_ll\\_and\\_lr\\_parsing](https://codemia.io/knowledge-hub/path/what_is_the_difference_between_ll_and_lr_parsing).
- GeeksforGeeks. (2025, 12 de julio). Difference between LL and LR parser.  
<https://www.geeksforgeeks.org/compiler-design/difference-between-ll-and-lr-parser/>.
- Kundel, D. (2020, 11 de junio). ASTs - What are they and how to use them. Twilio.  
<https://www.twilio.com/en-us/blog/developers/tutorials/building-blocks/abstract-syntax-trees>.
- Oreate AI Blog. (2026, 19 de enero). Decoding LL and LR Parsing: A Journey Through Syntax Analysis.  
<http://oreateai.com/blog/decoding-ll-and-lr-parsing-a-journey-through-syntax-analysis/bfe40a04adffb401707fde07a3fbe2cd>.
- Syntax (VII). (2025). Abstract Syntax Tree (cont.). CS333 Syntax Spring 2025. Colby College. [https://cs.colby.edu/courses/S25/cs333/notes/3.Syntax\(VII\).pdf](https://cs.colby.edu/courses/S25/cs333/notes/3.Syntax(VII).pdf).